
Final Project

The LunarLander Challenge

Reinforcement Learning

Format: closed and competitive. Every student solves the exact same environment under the exact same rules, and part of the grade is your performance *relative to the rest of the class* – not just whether your agent works, but how well it works compared to everyone else's. This document is self-contained: read all of it before writing any code, since several rules (the evaluation protocol above all) change what "a good solution" even means here.

1 The environment

Every student trains an agent on Gymnasium's **LunarLander-v2** – discrete actions (fire left engine, fire right engine, fire main engine, do nothing), an 8-dimensional continuous observation (position, velocity, angle, angular velocity, leg-contact flags), reward shaped for a smooth, centered, upright landing, large penalties for crashing, and a per-frame fuel cost. Gymnasium itself defines this environment as **solved** at an **average return of 200 or higher over 100 consecutive episodes** – that threshold, not a subjective judgment call, is the backbone of the grading formula below.

Why this environment for a closed/competitive project: it is small enough to train on a laptop CPU in well under an hour, but genuinely requires the function-approximation machinery from the second half of this course (a lookup-table agent cannot represent a continuous 8-D state space) – yet it is forgiving enough that a correct, reasonably-tuned implementation of a single course algorithm reliably solves it, which is exactly what makes ranking submissions against each other meaningful rather than a matter of luck.

2 Choose one track

	Track V (value-based)	Track P (policy-based)
Required algorithm	DQN, with experience replay and a target network (Lecture 10)	REINFORCE with a learned baseline, Actor-Critic, or PPO – your choice among these three (Lectures 11–13)
Action handling	Native (the environment's actions are already discrete)	Native (softmax over the 4 discrete actions)
Everyone in this track implements	The same required core loop	The same required core loop for whichever of the three you pick

You are ranked **only against other submissions in your own track** – Section 4's leaderboard component is computed separately per track, since a value-based and a policy-based agent are not doing directly comparable work. Track choice does not otherwise affect grading (neither track is worth more than the other).

Optional bonus (up to 5 extra points on the project, either track): also submit a working agent from the *other* track. It is graded on the same absolute-performance curve as Section 4.1 (not the competitive component, and not ranked) – this exists to reward genuine extra engagement with the course’s full breadth, not to double the required workload.

Hard constraints – violating any of these caps your score at 50/100 on this project regardless of performance

- **No RL library may provide the algorithm itself.** `stable-baselines3`, `RLlib`, `CleanRL`’s pre-built agents, and similar are off-limits for the core learning algorithm – the experience replay buffer, the network’s forward/loss/update, and the training loop must be your own code. `Gymnasium` (for the environment), `PyTorch/TensorFlow/JAX` (for tensors, autodiff, and optimizers), and `NumPy` are all fine and expected.
- **A fixed environment-step budget: 500,000 environment steps, total, across all your training runs for your primary submission.** Log your cumulative step count and report it. This exists so the competition is about algorithm quality and engineering, not about who has access to the most compute time – an agent that needs 2 million steps to solve the task has not actually out-performed one that does it in 300,000, whatever their final scores look like in isolation.
- **No hardcoding or seed-specific behavior.** Your policy must be a genuine function of the observation only. Grading is run on evaluation seeds that are not the ones you developed against (Section 3) – an agent that only works on seeds it has "seen" will score at or near the random-policy floor on the real grading run.

3 Evaluation protocol

Public seeds (yours to use freely): the 20 seeds

7404, 13783, 19113, 21818, 33091, 42191, 46784, 46841, 48180, 52582,

54637, 55510, 62379, 65851, 79439, 80265, 82779, 87379, 94258, 95232,

for your own training, validation, and the learning curves in your report.

Private seeds (held out): a separate, disjoint set of 100 evaluation seeds, not disclosed until after submission. Your submitted policy is loaded fresh and run once on each of the 100 private seeds with no further training; the average return over those 100 episodes is your **official evaluation score**. This mirrors a train/validation/test split – tune and pick your final checkpoint using the public seeds, but understand that a policy that has quietly overfit to quirks of the public seeds (rather than genuinely solving the task) will lose ground here, not gain it.

Seed variance in training, not just evaluation. Train your final submitted policy with **at least 3 different random seeds** for network initialization and exploration, and report the mean and standard deviation of each seed’s own evaluation performance (measured on the public seeds, before you ever see the private ones) in your report. Submit the checkpoint from whichever seed performed best on the public seeds – but a huge spread across your 3 seeds (some solving the task, some not) is itself something your report should discuss honestly, not hide: it’s a direct, personal replication of exactly the seed-sensitivity point already made with verified numbers in this course’s demo notebooks, and reporting it well is worth real credit even if your best single seed’s score is unremarkable.

3.1 Submission interface

Provide a single file `agent.py` exposing:

- `load_policy(checkpoint_path: str) -> Policy` – loads your trained weights.
- `Policy.select_action(observation: np.ndarray) -> int` – greedy/deterministic action selection (no exploration noise at evaluation time), returning one of the 4 discrete actions.

The grading harness imports this file, calls `load_policy` once, then calls `select_action` once per environment step across all 100 private-seed episodes. Any submission whose `agent.py` does not conform to this interface cannot be auto-graded and will be evaluated manually at a penalty (late-style deduction, per the course's standard policy) for the extra grading time it costs.

4 Grading

The project score is out of 100, split into two components, computed **separately per track** as noted above.

4.1 Absolute performance (70 of the 100 points)

A fixed curve from your official evaluation score (Section 3) to points – this component does not depend on anyone else's submission, so a well-implemented, well-tuned agent scores well here regardless of how the rest of the class does.

Official evaluation score (mean return over 100 private-seed episodes)	Points (of 70)
Below -50 (worse than a policy that mostly crashes)	0
-50 to 0, linearly	0 – 25
0 to 200 (the environment's own "solved" threshold), linearly	25 – 55
200 to 260 (a strong, well-tuned result), linearly	55 – 70
Above 260	70 (capped)

4.2 Competitive leaderboard (30 of the 100 points)

Your percentile rank on official evaluation score *within your track*, converted to points – this is the part of your grade that depends on your classmates' results, and the reason this project is described as competitive rather than pass/fail.

Percentile rank within your track	Points (of 30)
Top 10%	30
Next 25% (10th–35th percentile)	27
Middle 30% (35th–65th percentile)	22
Next 25% (65th–90th percentile)	16
Bottom 10%	10

Exception: if your official evaluation score is below the environment's own solved threshold (200), you receive **0** on the competitive component regardless of your rank – being "less bad than the bottom of the class" is not the same accomplishment as actually solving the task, and the grading should not reward it as if it were. (You can still, of course, earn up to 55/70 on the absolute component even without solving it.)

5 Deliverables

1. **Code:** your training notebook, submitted as an `.ipynb` that has **already been run top to bottom**, with every cell’s output (training-curve plots, logged returns, printed step counts) still attached – a notebook with its outputs cleared cannot be checked without re-running it and will be evaluated at a penalty for the extra grading time it costs. Also submit `agent.py` (the submission interface above) and your best checkpoint file.
2. **A short report (2-3 pages)**, covering: which track and algorithm you chose and why; your key hyperparameters and how you arrived at them (a brief note on what you tried, not just the final values); learning curves (return vs. environment steps) across your ≥ 3 training seeds, with the mean and spread reported honestly per Section 3; and your public-seed evaluation score prior to submission.
3. **Total environment steps used**, logged and reported, for the compute-budget check.

6 A worked sanity check

Before you trust your own training run, confirm your DQN implementation’s target computation against this one-transition example: state s , action a , reward $r = +5$, next state s' (non-terminal), $\gamma = 0.99$, and a target network giving $\max_{a'} Q_{\text{target}}(s', a') = 12.0$. The DQN target is

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a') = 5 + 0.99 \times 12.0 = 5 + 11.88 = 16.88.$$

If your loss is $(Q_{\theta}(s, a) - y)^2$ and your code computes anything other than 16.88 for this exact transition, debug the target computation before spending any of your 500,000-step budget on a run you can’t yet trust. (For Track P: the corresponding sanity check is that a REINFORCE gradient estimate on a single-step episode with reward r and policy probability $\pi_{\theta}(a | s) = 0.4$ for the action actually taken reduces to exactly $r \cdot \nabla_{\theta} \log 0.4$ for that one trajectory – verify your code reproduces the score-function term $\nabla_{\theta} \log \pi_{\theta}(a | s)$ correctly in isolation, the same debugging discipline used throughout this course’s own demo notebooks, before trusting a multi-episode training curve.)